BIT4

A Handcrafted 4-bit Computer Emulator

Technical Architecture & Developer Reference

Developer Notes

09.06.2026

Built with C# / WinForms — No emulation frameworks — Every layer hand-implemented

1. Overview

BIT4 is a fully custom 4-bit computer architecture implemented in C# and WinForms. Every hardware layer — CPU, ALU, RAM, ROM, VPU, assembler, and input system — is built from scratch with no emulation frameworks or shortcuts. The machine is architecturally authentic: all I/O is memory-mapped, the VPU uses a two-layer pixel/attribute design identical to 1980s home computers, and the instruction set is encoded in real 8-bit opcodes.

Design Philosophy

The constraint of 4 bits was intentional — to understand how early engineers extracted maximum functionality from minimal hardware. The result sits architecturally between the Intel 4004 (1971) and Intel 8080 (1974), with a VPU design closely matching the ZX Spectrum ULA and an exact CGA 16-color palette. The stack-based CPU design mirrors Forth machines of the late 1970s, considered one of the most elegant minimal CPU architectures.

Estimated Transistor Equivalent

Component	Transistors
Registers (AX, BX)	~90
ALU (16 operations)	~678
Stack (LIFO, 8 levels)	~320
RAM (16 x 4-bit SRAM cells)	~384
Character ROM (64 chars)	~2,560
VPU (pixel buffer + attribute matrix)	~1,030
Input port + glue logic	~390
TOTAL	~5,452

This places BIT4 between the Intel 8008 (3,500 transistors, 1972) and Intel 8080 (6,000 transistors, 1974) in equivalent complexity.

2. CPU Architecture

Registers

Register	Width	Purpose
AX	4-bit	Primary general purpose register
BX	4-bit	Secondary general purpose register
PC	32-bit int	Program counter — tracks current execution line
ZeroFlag	1-bit bool	Set when last ALU result equals 0000
CallStack	Stack<int>	Return address stack for CALL/RET instructions

Stack

The CPU is stack-based. All ALU operations pop operands from the stack and push results back. MOV loads values into AX/BX registers, PUSH puts them on the stack, and POP retrieves them. This is a LIFO structure — the last value pushed is the first popped. Binary operations pop B first (top), then A (second), matching standard stack convention.

Execution Pipeline

1. Compile() — scans all lines, builds label dictionary, resets PC to 0
2. ExecuteNextLine() — called per clock tick by the WinForms timer
3. Skip empty lines and label declarations
4. Check for control flow instructions (JMP, JZ, JNZ, CALL, RET)
5. Otherwise pass line to RunCode() for instruction dispatch
6. RunCode() handles MOV, PUSH, POP, LOAD, STORE, PRINT, then ALU ops
7. PC increments after each normal instruction

3. Instruction Set Reference

Data Movement

Mnemonic	Encoding	Operation	Stack Effect
MOV AX, xxxx	1100xxxx	Load 4-bit immediate into AX	None
MOV BX, xxxx	1101xxxx	Load 4-bit immediate into BX	None
PUSH AX	11000000	Copy AX onto stack top	+1
PUSH BX	11010000	Copy BX onto stack top	+1
POP AX	11100000	Move stack top into AX	-1
POP BX	11110000	Move stack top into BX	-1

Memory Operations

Mnemonic	Encoding	Operation	Stack Effect
LOAD 0x??	0110xxxx	Read RAM[addr], push value to stack	+1
STORE 0x??	0111xxxx	Pop stack, write to RAM[addr]	-1

ALU — Logical Operations (Binary)

Mnemonic	Opcode	Operation	Stack Effect
AND	0000	Bitwise AND of top two stack values	-1 (result pushed)
OR	0001	Bitwise OR of top two stack values	-1 (result pushed)
XOR	0010	Bitwise XOR of top two stack values	-1 (result pushed)
NOT	0011	Bitwise NOT of stack top (unary)	0 (result pushed)

ALU — Arithmetic Operations

Mnemonic	Opcode	Operation	Type
ADD	0100	4-bit addition, wraps at 1111	Binary
SUB	0101	4-bit subtraction	Binary
INC	1010	Increment top of stack by 1	Unary
DEC	1011	Decrement top of stack by 1	Unary
SHL	1000	Shift left (multiply by 2)	Unary
SHR	1001	Shift right (divide by 2)	Unary

Control Flow

Mnemonic	Encoding	Condition	Action
JMP label	11111111	Always	PC = label line

Mnemonic	Encoding	Condition	Action
JZ label	11111110	ZeroFlag == true	PC = label line
JNZ label	11111011	ZeroFlag == false	PC = label line
CALL label	11111101	Always	Push PC+1 to CallStack, PC = label
RET	11111100	Always	PC = CallStack.Pop()

VPU Output

Mnemonic	Encoding	Operation
PRINT	11110111	Pop stack, look up char ID in active ROM page, draw 4x5 bitmap to screen at cursor position

4. Memory Map

BIT4 uses a 16-address memory space (0x00–0x0F). Addresses 0x00–0x07 are general purpose RAM. Addresses 0x08–0x0F are hardware-intercepted ports — reading or writing them triggers real hardware actions instead of normal RAM access. This is identical to how real hardware does memory-mapped I/O.

Address	Dec	Port Name	Direction	Function
0x00–0x07	0–7	General RAM	R/W	Free-use data storage
0x08	8	INPUT PORT	Read only	Keyboard state — 4-bit key code
0x09	9	ROM PAGE	Write	Switch active character ROM bank (0–3)
0x0A	10	CLS	Write	Clear screen, reset cursor and color
0x0B	11	NEWLINE	Write	Cursor Y-jump, reset X to 0
0x0C	12	FAST FWD	Write	Cursor X-skip without drawing
0x0D	13	VSYNC / HOME	Write	Teleport cursor to origin (0, 0)
0x0E	14	COLOR	Write	Set active palette color (0–15)
0x0F	15	DRAW	Write	Stream 4 pixels to screen, advance cursor

Hardware Intercept Pattern

STORE to address 0x09 does not write to RAM — it intercepts the value and passes it to the VPU ROM page selector. STORE to 0x0F intercepts and passes the 4-bit payload to the screen draw hardware. LOAD from 0x08 intercepts and returns the current keyboard state instead of RAM data. All other addresses perform normal RAM read/write.

5. VPU — Video Processing Unit

Display Architecture

The VPU uses a two-layer architecture matching the ZX Spectrum ULA design. Layer 1 is a monochrome pixel buffer (512x512 boolean array). Layer 2 is a color attribute matrix (4096 cells, 64x64 grid of 8x8 pixel blocks). Color is applied per-block, not per-pixel — this is authentic to real 1980s hardware and produces the characteristic "attribute clash" of that era.

Display Specifications

Property	Value
Resolution	512 x 512 pixels
Pixel scale	4x4 macro-pixels (each logical pixel = 4x4 screen pixels)
Color cells	4096 (64 x 64 grid of 8x8 pixel blocks)
Palette	16 colors (exact CGA)
Character size	4 x 5 pixels (logical), 16 x 20 screen pixels
Chars per row	25 characters (512 / (5 * 4) = ~25)

CGA Color Palette

Code	Binary	Color Name	RGB Value
0	0000	Black	#000000
1	0001	Dark Blue	#0000AA
2	0010	Dark Green	#00AA00
3	0011	Dark Cyan	#00AAAA
4	0100	Dark Red	#AA0000
5	0101	Dark Magenta	#AA00AA
6	0110	Brown	#AA5500
7	0111	Light Gray	#AAAAAA
8	1000	Dark Gray	#555555
9	1001	Bright Blue	#5555FF
10	1010	Bright Green	#55FF55
11	1011	Bright Cyan	#55FFFF
12	1100	Bright Red	#FF5555
13	1101	Bright Magenta	#FF55FF
14	1110	Bright Yellow	#FFFF55
15	1111	Pure White	#FFFFFF

6. Character ROM

The character ROM stores 64 bitmap characters across 4 pages. Each character is a 4x5 pixel boolean array. The active page is selected by writing a 4-bit page index to the ROM page port (STORE 0009). PRINT pops a 4-bit character ID (0–15) from the stack and draws the corresponding bitmap at the current cursor position.

Page	Port Value	Binary	Character Set
0 (Data)	0	0000	A B C D E F G H I J K L M N O P
1 (Data1)	1	0001	Q R S T U V W X Y Z [\ ] ^ _ (space)
2 (DataArithSymbols)	2	0010	! " # $ % & ' ( ) * + , - . / ?
3 (DataDigits)	3	0011	0 1 2 3 4 5 6 7 8 9 : ; < = > >

Switching Pages — Assembly Example

```
; Switch to ROM page 1 (Q-Z)
mov ax,0001
push ax
store 0009 ; hardware intercept: sets ActiveRomPage = 1

; Print R (ID 1 on page 1)
mov ax,0001
push ax
print
```

7. Keyboard Input

Keyboard state is read from the hardware input port at address 0x08. The WinForms form captures KeyDown and KeyUp events and writes the current key code to the InputPort static class. LOAD 0008 is intercepted — instead of reading RAM, it returns the current key state as a 4-bit value pushed onto the stack.

Code	Binary	Key
0	0000	No key pressed
1	0001	UP arrow
2	0010	DOWN arrow
3	0011	LEFT arrow
4	0100	RIGHT arrow
5	0101	SPACE
6	0110	Z
7	0111	X

Reading Input — Assembly Example

```
loop:
load 0008 ; read keyboard port into stack
mov ax,0001 ; UP arrow key code
push ax
xor ; XOR: result = 0000 only if key == UP
jz up_pressed ; jump if zero flag set
pop ax ; clean XOR result from stack
jmp loop

up_pressed:
pop ax
; ... handle UP key ...
jmp loop
```

8. Assembler

The assembler is a single-pass interpreter with a label pre-scan. There is no separate compile-to-binary step — instructions execute directly from the mnemonic text. Machine code encoding is provided for UI display purposes only.

Execution Model

Compile() does a full scan to build the label dictionary. ExecuteNextLine() is called once per timer tick. The program counter (PC) advances after each normal instruction. Jump instructions set PC directly without incrementing. Labels are skipped during execution — they only exist in the label dictionary.

Label Syntax

```
myloop: ; label declaration (ends with colon)
load 0001 ; normal instruction
jmp myloop ; jump to label
```

Subroutine Pattern

```
call print_char ; push PC+1, jump to print_char
jmp done

print_char:
mov ax,0000
push ax
print
ret ; pop return address, jump back

done:
```

9. Historical Comparison

BIT4 was designed as a learning exercise in minimal hardware design. The constraints — 4-bit bus, 16 bytes RAM, memory-mapped I/O — mirror real engineering constraints of the early 1970s. The result is a machine with authentic architectural characteristics rather than a simplified simulation.

Feature	BIT4	Intel 4004 (1971)	Atari 2600 (1977)	ZX Spectrum (1982)
Data bus	4-bit	4-bit	8-bit	8-bit
RAM	16 bytes	640 bytes	128 bytes	48 KB
ROM	64 chars/4 pages	Paged ROM	Cart ROM	16 KB
CPU style	Stack-based	Accum.	Accum.	Accum.
Color system	Attribute matrix	None	Scanline	Attr. matrix
Palette	16 (CGA)	None	128	16
Mem-mapped I/O	Yes	No	Yes (TIA)	Yes (ULA)
Separate VPU	Yes	No	Yes (TIA)	Yes (ULA)
Transistors	~5,452	2,300	N/A	N/A

The stack-based CPU is architecturally closer to Forth machines of the late 1970s than to mainstream processors. The VPU pixel+attribute dual-layer design is nearly identical to the ZX Spectrum ULA. The CGA 16-color palette is an exact match to the IBM CGA card (1981).

10. Project Structure

File	Responsibility
Assembler.cs	Parser, instruction executor, program counter, label memory, JMP/JZ/JNZ/CALL/RET
AluInputBuffer.cs	ALU input staging, opcode dispatch to Alu.cs
Alu.cs	Core ALU operations: AND, OR, XOR, NOT, ADD, SUB, SHL, SHR, INC, DEC
Register.cs	4-bit register model with bool[4] array
DataMemory.cs	16-byte RAM array + hardware port intercepts
ComputerMonitor.cs	VPU: pixel buffer (512x512), color attribute matrix (4096), cursor, DrawCharacter()
CharacterRom.cs	4-page character bitmap ROM, 64 total characters
HardwarePalette.cs	CGA 16-color palette as Color[] array
InputPort.cs	Keyboard hardware input port, SetKey() / ReadKey()
Form1.cs	WinForms UI, rendering loop, KeyDown/KeyUp events, timer

BIT4 Emulator — Developer Notes — 09.06.2026

Built with C# / WinForms — No emulation frameworks — Every component hand-implemented